

Proyecto de Transformaciones Geométricas

1) Deformaciones lineales

Obtención de la matriz P de una transformación proyectiva

La función `get_afin.m` suministrada halla la matriz P de una transformación afín a partir de las coordenadas origen (x,y) y destino (u,v). Las coordenadas (x,y,u,v) se pasan a la función como vectores fila. Si miráis el código veréis cómo la función construye la matriz M que vimos en clase a partir de (x,y) y usando dicha matriz resuelve un sistema lineal cuya solución son las dos primeras filas de la matriz P (la 3ª fila de una transformación afín es siempre 0 0 1). Notad que el código no asume nada sobre el número de puntos usado (tamaño de x,y, ...) y usa \ para resolver el sistema, por lo que va a funcionar tanto en el caso de tener el mínimo de puntos necesarios (3) cómo si disponemos de más puntos (en cuyo caso la transformación será el mejor ajuste). Probad la función usando `x=[0 600 465]; y=[0 65 680]; u=[275 655 365]; v=[30 285 755];`

Para verificar que la transformación P obtenida es correcta escribiremos una función `[u,v]=convertir(x,y,P)`. La función recibe las coordenadas x,y (1xN) en el espacio de partida y les aplica la transformación indicada por la matriz P devolviendo las coordenadas de dichos puntos (u,v también 1xN) en el espacio destino. Para ello:

- Cread una matriz de coordenadas homogéneas apilando las coordenadas x's (1ª fila), las y's (2ª fila) y añadiendo una 3ª fila con 1's.
- Multiplicad P por esta matriz de coordenadas homogéneas, obteniendo una nueva matriz también de tamaño 3xN.
- Las coordenadas de salida u,v (1 x N) serán las dos primeras filas de la matriz resultante, DIVIDIDAS PUNTO a PUNTO por la tercera fila.

Si hacéis ahora `[u2,v2]=convertir(x,y,P)`, las nuevas coordenadas obtenidas deben ser idénticas a las coordenadas u,v iniciales, ya que por definición P se construye para pasar de las coordenadas (x,y) a (u,v).

Llamad ahora a `get_afin()` cambiando (x,y) por (u,v) y [usad la función `show\_mat\(A\)` para volcar la transformación iP resultante \(que pasa de coordenadas u,v a x,y\)](#). Comprobad que iP es la inversa de la matriz P original (x,y → u,v), confirmando que en el caso lineal la matriz de la transformación inversa corresponde a la inversa de la matriz de la transformación original.

Partid de `get_afin()` y modificadla para escribir `function P=get_proy(x,y,u,v)`, que calcule ahora la matriz de la transformación proyectiva que convierta (x,y) en (u,v). Seguid las indicaciones de las transparencias para construir la matriz M del sistema lineal y cuya solución son los coeficientes de la correspondiente matriz proyectiva. Los argumentos (x,y) e (u,v) son como antes vectores fila con las coordenadas de (al menos) 4 puntos de origen y destino.

Probad la función con las coordenadas $x=[275 \ 655 \ 365 \ 25]$; $y=[30 \ 285 \ 755 \ 340]$; $u=[0 \ 600 \ 465 \ 215]$; $v=[0 \ 65 \ 680 \ 585]$; Volcad la matriz P obtenida con `show_mat`.

De nuevo, verificar que P es correcta con la función `convertir()`. ¿De qué orden son ahora las diferencias entre las coordenadas u,v originales y las obtenidas al aplicar P a (x,y)? Adjuntad código de vuestra función `get_proy` una vez verificada. ¿A qué coordenadas u,v iría a parar un punto con coordenadas (x=200,y=100) al aplicarle la transformación proyectiva dada por P?

Volcad ahora la matriz obtenida con `get_proy(u,v,x,y)`, que pasa de coordenadas destino (u,v) a origen (x,y). Volcad también la matriz inversa (inv) de la matriz P anterior ($x,y \rightarrow u,v$). ¿Son iguales?

Deformación de una imagen usando transformaciones lineales

Vamos a escribir una función `warp_img()` que aplique la transformación lineal dada por la matriz P a la imagen im (con valores entre 0 y 1) y que devuelva la imagen deformada: `function im2=warp_img(im,iP)`

La función recibe la imagen im (con valores de tipo double entre 0 y 1) y la matriz iP (3x3) con **la inversa** de la transformación P que se desea aplicar a im. Usamos la matriz inversa porque, como vimos en clase, para hallar la imagen deformada (im2) es habitual implementar un "warping" inverso: se barren las coordenadas de destino en im2 y se aplica P^{-1} para ver donde caen esas coordenadas en la imagen original.

Como las coordenadas transformadas no caerán exactamente sobre píxeles enteros de la imagen original tendremos que hacer una interpolación. Para interpolar se usará la función `interp2`: `A_xy=interp2(A,x,y,tipo_int)`, que recibe una matriz A (2D) cuyas coordenadas asumimos que van de 1 a N (alto imagen) para las Y's y de 1 hasta M (ancho imagen) para las X's. La función devuelve el valor interpolado (A_xy) en las coordenadas (x,y) especificadas (posiblemente no enteras). Si las coordenadas (x,y) son un único valor el resultado es el valor de A interpolado para ese punto. Los argumentos (x,y) pueden ser también un par de vectores o matrices. En ese caso el resultado es un vector o matriz con los valores interpolados en todos los pares de coordenadas dadas. El último parámetro indica el tipo de interpolación ('bicubic', 'bilinear', ...) a usar.

Dentro de la función `warp_img()` debéis:

- 1) Determinar el alto N, ancho M y número de planos de color de la imagen im. Con la función `zeros()` reservar espacio para la imagen de salida im2 (mismo tamaño que la original). Reservad también dos matrices X e Y (2D) de tamaño NxM para guardar en ellas las coordenadas donde queremos hacer la interpolación.
- 2) Para rellenar las coordenadas a interpolar X e Y recorreremos las coordenadas destino (u,v) en un doble bucle (desde 1 a M para u y de 1 a N para v) y usando

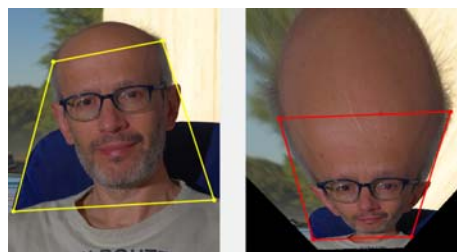
convertir() veremos donde caen (x,y) en la imagen de partida al aplicarles la transformación inversa (iP). Guardad la coordenada x obtenida en la casilla (v,u) de la matriz X y haced lo mismo para la coordenada y.

- 3) Completado el bucle y rellenas las matrices X, Y con las coordenadas a interpolar solo queda usar interp2 para que MATLAB haga la interpolación en esos puntos (usad 'bicubic' como tipo de interpolación). **Como interp2 trabaja solo con matrices 2D tendremos que hacer un bucle interpolando cada plano de color de im por separado y guardando el resultado en el correspondiente plano de im2.**

Adjuntad código de warp_img.

Cargad la imagen del fichero "foto.jpg" y convertirla a tipo double con valores entre 0 y 1. Deformad esta imagen aplicando la transformación P del apartado anterior (recordad que para aplicar la transformación P le pasamos a la función warp_img la matriz inversa P^{-1}). Adjuntad la imagen resultante. Veréis zonas en negro que corresponden a puntos que caen fuera del soporte de la imagen original, para los que Matlab devuelve el valor de NaN. También habrá partes de la imagen original que no aparecen en la imagen deformada. La razón es que al hacer el bucle en (u,v) hemos barrido un rectángulo con el mismo "tamaño" y posición que la imagen original. Dependiendo de la transformación usada, algunas partes de la imagen deformada pueden caer fuera de esa zona.

Tras completar estas funciones podréis usar el programa demoP4 con `>>demoP4`. El programa muestra la imagen origen (a la izquierda) y destino (a la derecha) y sobre



ellas los cuadriláteros que definen la deformación. Pinchando con el ratón en sus vértices podéis mover sus esquinas y deformarlos (cada vez que se cambian el programa vuelca sus nuevas coordenadas). Luego el programa llama a vuestra rutina get_proy con dichas coordenadas para calcular la matriz iP y usarla para deformar la imagen (usando warp_img).

Llamando a la función con demoP4(im) podréis usar cualquier otra imagen en vez de la mía. Usando una foto vuestra, tratad de obtener para la imagen deformada un resultado similar al que se muestra en la figura anterior. Partid de una fotografía vuestra en modo retrato (lado largo de la foto en vertical) con una resolución de unos 900x600 o similar (podéis usar las funciones imcrop e imresize de MATLAB para ajustar la foto de partida a estas dimensiones). Adjuntad captura de demoP4 donde se vea la imagen original y la deformada. Volcad las coordenadas de los vértices elegidos y calculad la matriz P que define la deformación mostrada.

Tras determinar P aplicar vosotros directamente la deformación correspondiente a P a vuestra imagen y adjuntad captura del resultado. Debéis obtener algo muy similar a lo mostrado a lo que muestra demoP4 en la imagen de su derecha.

Imágenes recursivas

Como una transformación proyectiva nos permite transformar cualquier cuadrilátero en otro, puede usarse para insertar una fotografía (un rectángulo) dentro de otra superficie plana en una segunda foto (que debido a la perspectiva se verá como otro cuadrilátero). En este caso las coordenadas de partida serían las esquinas de la 1ª foto y las de llegada, las del cuadrilátero destino en la 2ª foto.



Usando la misma imagen como origen y destino podemos crear fotos "recursivas", insertando sucesivas copias de una imagen dentro de sí misma. Por ejemplo, podemos usar una foto en la que aparece un cuadro o similar, dentro del cual se inserta la imagen original (donde a su vez aparece de nuevo el cuadro, ...)



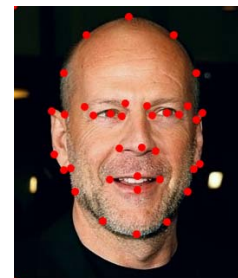
Se trata de usar las funciones desarrolladas hasta ahora en la práctica para crear una imagen recursiva a partir de la foto de "billboard.jpg":

- El objetivo es hallar una transformación proyectiva que, aplicada a la imagen completa, la proyecte dentro del espacio del cartel que aparece en la propia imagen. [Volcad \(show_mat\) la matriz P de dicha transformación](#). Usando `warp_img()` deformad la imagen de acuerdo a esta transformación y [adjuntad la imagen resultante](#).
- Para fundir ambas imágenes usad el hecho de que la imagen deformada tiene valores NaN en las zonas que caen fuera de la valla publicitaria. De esta forma se puede saber en qué puntos de la imagen final hay que usar los datos de la imagen original y en cuáles los de la imagen deformada. Se valorará hacerlo sin bucles. [Adjuntad código usado para fusionar ambas imágenes y captura de la imagen final resultante](#).
- En la nueva imagen aparecerá un nuevo cartel publicitario dentro del espacio del antiguo. [¿Se os ocurre cómo repetir el proceso para insertar copias adicionales de la foto dentro del nuevo cartel creado SIN TENER QUE VOLVER A MEDIR más coordenadas?](#)

2. Aplicación: Morphing

En este apartado implementaremos el proceso de *morphing* visto en clase entre las imágenes 'willis.jpg' y 'ant.jpg'. Cargadlas como im1 e im2 y convertirlas a double con valores entre 0 y 1. Sobre estas imágenes he marcado varios puntos de control.

Haciendo `load puntos_morph` veréis 4 vectores fila (x_1, y_1, x_2, y_2) de tamaño 1×40 con las coordenadas de 40 puntos de control. En (x_1, y_1) tenéis las coordenadas sobre la primera imagen (willis) y en (x_2, y_2) las coordenadas de los mismos puntos en la 2ª imagen (ant). Si visualizáis una imagen y superponéis sobre ella (hold on) sus puntos de control debéis ver una figura similar a la adjunta.

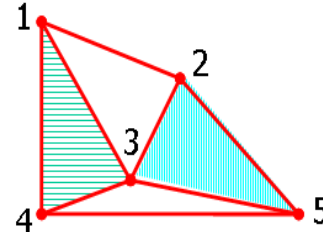


a) Triangulación y cálculo de las matrices de transformación

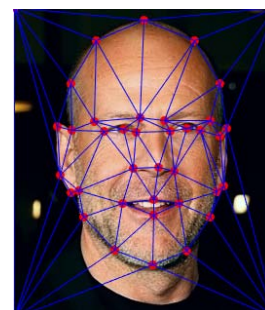
Calculad la media de los puntos de control en ambas imágenes con: $u = 0.5 \cdot (x_1 + x_2)$
 $v = 0.5 \cdot (y_1 + y_2)$

Estas posiciones medias (u, v) son los puntos a donde queremos deformar tanto los puntos de la primera imagen (x_1, y_1) como los de la segunda (x_2, y_2). Usaremos estos puntos (u, v) para calcular una malla triangular cubriendo las fotos y cuyos vértices sean dichos puntos. Para calcularla usaremos el algoritmo de Delaunay: `T=delatunay(u,v)`. El algoritmo recibe la lista de puntos en dos vectores (u, v) y devuelve una matriz T (con un tamaño $NT \times 3$) describiendo los NT triángulos de la triangulación. Cada fila de T son los tres índices de los vértices de cada triángulo referidos a la lista de vértices en (u, v). Por ejemplo, los cuatro triángulos de la triangulación mostrada en la figura adjunta se codificarían a partir de sus cinco vértices como:

```
T = [ 1 3 4    % vertices 1, 3 ,4 = triángulo rallado
      2 3 5    % vertices 2, 3 ,5 = triángulo azul
      3 4 5    ...
      1 2 3]   ...
```



¿Cuántos triángulos tiene la triangulación obtenida? Llamad NT a ese número. Tras calcular la triangulación podemos visualizarla con el comando `tripplot(T,x,y)` y superponerla sobre la imagen anterior de la foto y los puntos de control (ver la figura adjunta). Adjuntad una captura de la segunda foto mostrando sus puntos de control y la triangulación. Recordad que en cada imagen se usan sus propios puntos de control pero que la triangulación T es única (obtenida a partir de los puntos medios u, v).



El paso siguiente es calcular las transformaciones entre todos los triángulos de cada imagen (definidos por sus coordenadas x, y + triangulación) y sus correspondientes triángulos en la imagen central (definidos por u, v). Al tener una matriz PARA CADA UNO DE LOS TRIÁNGULOS que componen la triangulación habrá que guardar NT

matrices (3x3) de transformación. Al trabajar con triángulos, las matrices buscadas corresponderán a transformaciones afines. Al igual que antes, para deformar las imágenes usaremos las transformaciones inversas, las que pasan de coordenadas u,v (imagen destino) a las correspondientes coordenadas (x,y) sobre las imágenes originales.

Para la primera imagen, estas matrices las guardaremos en un *array de celdas* creado con `iP1=cell(1,NT)`. Vimos que un array de celdas es una generalización de una tabla en cuyas casillas podemos guardar cualquier objeto. La única diferencia es que para acceder a sus "casillas" usaremos llaves en vez de paréntesis. En este caso si se desea acceder a la matriz de la k -ésima transformación (para modificarla o para operar con ella) lo haremos con `iP1{k}`. Para calcular estas NT matrices haremos un bucle en k desde 1 a NT (número de triángulos) y en cada paso:

- 1) Extraemos los índices de los vértices del k -ésimo triángulo (guardados en la k -ésima fila de la matriz T de la triangulación).
- 2) Usando esos índices extraemos de $x1$ y $y1$ las coordenadas de los tres vértices de origen en la 1ª imagen y las guardamos en X,Y . Igualmente sacamos de u,v las coordenadas de los tres vértices de destino (U,V) .
- 3) Usamos `get_afin()` para hallar la transformación que pasa de coordenadas (U,V) a (X,Y) , guardando la matriz 3x3 obtenida en `iP1{k}`.

Repetir para obtener las matrices iP2 correspondientes a la 2ª imagen, que transforman los triángulos de la segunda imagen en los correspondientes triángulos de imagen central. Para ello podéis aprovechar el mismo bucle: una vez calculada `iP1{k}`, extraemos (ahora de $x2,y2$) las nuevas coordenadas de origen (X,Y) para la 2ª imagen. Notad que las coordenadas de destino (U,V) no cambian, ya que se desea deformar ambas imágenes al las mismas posiciones de destino. Repitiendo la llamada a `get_afin` obtendréis la nueva transformación que guardaréis en `iP2{k}`. [Adjuntad código del bucle para calcular las matrices de transformación iP1 e iP2.](#)

[Volcad en la hoja de respuestas \(usando `show\_mat`\) las matrices obtenidas para iP1 e iP2 correspondientes al triángulo nº 3 de la triangulación.](#)

Para deformar una imagen aplicando este tipo de transformaciones en el siguiente apartado habrá que conocer en qué triángulo de la imagen destino cae cada píxel. Para ello usaremos la rutina suministrada `determina_triang()` que calcula (para todos los píxeles de la imagen) el triángulo al que pertenecen. La rutina se llama (una sola vez) como:

```
zona=determina_triang(T,u,v,N,M);
```

La función recibe los datos de la triangulación (T,u,v) y el tamaño de las imágenes $(N \times M)$ y devuelve una matriz **zona** (también $N \times M$). El valor `idx=zona(i,j)` nos da el índice del triángulo (1,2,3...,NT) en el que cae el píxel (i, j) de la imagen destino.

Dicho píxel deberá transformarse con la matriz $iP1\{idx\}$ o $iP2\{idx\}$ (dependiendo de si estamos deformando la primera o la segunda imagen)

b) Escribir una rutina para deformar una imagen aplicando la transformación a trozos dada por la familia de matrices $iP\{\}$:

```
function im=warp_img_trozos(im,iP,zona)
% im = imagen a deformar
% iP = familia de matrices (afines) a utilizar
% zona = información del triángulo de cada píxel.
```

La función es muy similar a `warp_img` del apartado anterior. Las diferencias son que:

- La función recibe un parámetro adicional (zona) con la información de en qué triángulos caen los distintos píxeles de la imagen.
- El segundo parámetro iP , en vez de una matriz (inversa de la transformación a aplicar), es ahora una colección de matrices (las inversas de las matrices que definen la deformación de cada triángulo).

Para el código de la función podéis partir del código de `warp_img()`, ya que apenas hay que modificarlo. Dentro del bucle (en u,v) donde se rellenaban las matrices X e Y , antes de llamar a `convertir()` para que transforme las coordenadas (u,v) a (x,y) hay que determinar qué transformación usar, dependiendo del triángulo donde cae el píxel (u,v) donde nos encontramos. Consultad el correspondiente valor de zona y usadlo para elegir la transformación adecuada de entre la lista de matrices $iP\{\}$.

[Adjuntad las líneas de código modificadas.](#)

Usad la nueva función para deformar las dos imágenes a su posición media (usando las transformaciones inversas de $iP1$ para la primera imagen y las de $iP2$ para la segunda). [Adjuntar las dos imágenes deformadas.](#) Finalmente haced su media para obtener la mezcla final y [adjuntad una captura del resultado.](#)

Es sencillo modificar el código para calcular la fusión en cualquier otro punto que no sea exactamente el punto medio entre las imágenes. Para ello, al hacer la media de los puntos x 's e y 's para obtener los puntos destino (u,v) , haremos que dependan de un valor t (entre 0 y 1):

$$u = (1 - t) \cdot x_1 + t \cdot x_2$$

$$v = (1 - t) \cdot y_1 + t \cdot y_2$$

De esta forma podemos mover gradualmente los puntos de la malla destino (u,v) desde su posición en la 1ª imagen ($t=0 \rightarrow u=x_1, v=y_1$), hasta su posición en la 2ª imagen ($t=1 \rightarrow u=x_2, v=y_2$), pasando por el punto medio de antes ($t=1/2$).

Al cambiar (u,v) debemos recalcular las matrices $iP1$ e $iP2$ de las transformaciones y volver a llamar a la función `determina_triangu` (al depender ésta de u y v).

La triangulación T no la cambiaremos, usando siempre la calculada en el apartado anterior para los puntos medios.

Tras recalcular $iP1$, $iP2$ y zona para el valor de t deseado, usad `warp_img_trozos()` para deformar las dos imágenes. El resultado será ahora una media ponderada usando también el valor de t :

$$\text{res} = (1-t) \cdot \text{im1_d} + t \cdot \text{im2_d}$$

Adjuntad las imágenes deformadas y la mezcla obtenida para $t=0.7$, junto con todo el código usado para crearla (sin las funciones ya incluidas).

c) Optimización de la rutina `warp_img_trozos()`

En la función anterior el cuello de botella no es la función de interpolación (`interp2`) sino el cálculo previo de las coordenadas X e Y para todas las posiciones (u,v) de la imagen. Para optimizar el proceso se puede cambiar los dos bucles en (u,v) por un solo bucle desde $k=1$ hasta NT barriendo los triángulos. En cada paso se extraen las coordenadas de todos los píxeles que caen dentro de cada triángulo y como a todos esos puntos les afecta la misma transformación se pueden procesar conjuntamente.

Para procesar las coordenadas (u,v) de todos los puntos pertenecientes al k -ésimo triángulo:

- Hallar los puntos que pertenecen al k -ésimo triángulo `pt=find(zona==k);`
- Haciendo `[v,u]=ind2sub(size(zona),pt')` obtenemos las coordenadas (u,v) de dichos puntos.
- Usando `convertir()` transformamos (u,v) en coordenadas (x,y) usando la correspondiente matriz inversa.
- Finalmente, para guardar las coordenadas (x,y) obtenidas en las matrices X,Y basta hacer: `X(pt)=x; Y(pt)=y;`

Adjuntad código de vuestra función optimizada.

Usando `tic/toc` medir su tiempo de ejecución para deformar 100 veces una imagen y compararlo con el de la versión no optimizada. Volcad los tiempos obtenidos.

d) Morphing con vuestras propias imágenes

Vamos a repetir el proceso de morphing pero usando ahora una foto vuestra y de vuestro compañero/a de prácticas (si sois tres en el grupo combinaremos una transición de `alumno1` a `alumno2` y luego de `alumno2` a `alumno3`). Las imágenes usadas deben ser del mismo tamaño y cuanto más parecidas sean mejor será su mezcla. Intentad que la escala y orientación de las caras en las fotos sea similar, evitad que una imagen sea muy clara y la otra muy oscura, etc. Lo mejor es haceros un par de fotos con el mismo móvil en el mismo lugar para tener un fondo similar. Un fondo sin detalles (cielo, pared uniforme, ...) suele ser preferible. El código que tenéis debería funcionar con cualquier tamaño de imágenes, pero puede ser lento si son grandes. Si es necesario podéis usar un software como `IrfanView` para cambiar su tamaño antes de procesarlas.

Obviamente para estas imágenes no valdrán los puntos de control que os di antes. Con la aplicación `marcar_puntos` podéis seleccionar puntos de control en vuestras fotos. Su uso es: `marcar_puntos("foto1.jpg","foto2.jpg")`. El programa carga las dos imágenes y las muestra en una figura. Pinchando con el ratón (botón izquierdo) podéis marcar puntos en una u otra imagen. Podéis pinchar alternativamente en una y otra imagen o marcar varios puntos en una de ellas y luego marcarlos en el mismo orden en la otra (siempre **en el mismo orden** en ambas imágenes). Poned unos 20/30 puntos en cada imagen usando los ojos, nariz, orejas, contorno de la cara, de forma similar a los puntos de control de las imágenes "test". Cuando terminéis pinchad el botón derecho. El programa añadirá automáticamente las coordenadas de las 4 esquinas a vuestros puntos y guardará la información en un fichero llamado `puntos.mat`. Al terminar, si hacéis `>>load puntos`, veréis como antes las coordenadas de los NP puntos de control seleccionados en 4 vectores columna (x_1, y_1, x_2, y_2) de tamaño $NP \times 1$. [Adjuntad las imágenes originales con los puntos de control que habéis marcado superpuestos sobre ellas.](#)

Con estos puntos y vuestras imágenes es simplemente una cuestión de volver a correr el código anterior para reproducir sus resultados. [Adjuntad la imagen mezcla \(imágenes si sois más de 2\) para el punto medio \(\$t=0.5\$ \) entre cada pareja de fotos originales.](#)

Transición desde una imagen a otra.

Una vez que sabemos hacer la "mezcla" para cualquier valor de t , la calcularemos para una secuencia de valores desde $t=0$ a $t=1$ y vuelta a $t=0$. Usad el comando siguiente para crear un vector t con 40 puntos: `t=[(0:0.05:1) (0.95:-0.05:0.05)];`

Luego, con un bucle, calculad la imagen mezcla para cada $t(k)$ y guardadlas en una serie de ficheros llamados por ejemplo `mezcla01.jpg`, `mezcla02.jpg`, ..., haciendo:

```
fich=sprintf('mezcla%02d.jpg',k); imwrite(res,fich,'Quality',95);
```

Subid las imágenes obtenidas a <https://imgflip.com/gif-maker> para convertirlas en un GIF, similar al que os adjunto (ejemplo.gif) para la transición entre las imágenes test del apartado anterior). Si sois tres en el grupo incluid las transiciones: `AL1 -> AL2 -> AL3 -> AL1`.

[Subid el GIF resultante en la entrega.](#)

3) Transformaciones no lineales y sus inconvenientes

Las transformaciones lineales usadas en los apartados anteriores (definidas con una matriz 3×3) son las más habituales, pero también es posible usar cualquier otra aplicación $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ para pasar de un conjunto de coordenadas (x,y) a otro (u,v) .

En este apartado exploraremos una sencilla transformación no lineal, dada por:

$$u = A + B \cdot x + C \cdot y + D \cdot xy$$

$$v = a + b \cdot x + c \cdot y + d \cdot xy$$

La transformación es muy similar a una transformación afín pero añadiéndole un término adicional (coeficientes d y D) donde introducimos la no-linearidad como el producto $x \cdot y$. Esta transformación tiene el mismo número de coeficientes (8) que una proyectiva y, como en aquel caso, necesitaremos como mínimo 4 puntos para determinarla (ya que cada punto aporta 2 ecuaciones).

Lo primero es escribir una función `function P=get_nolineal(x,y,u,v)` (equivalente a `get_proy` o `get_afin` que reciba las coordenadas origen (x,y) y destino (u,v) como vectores $1 \times N$. Tras calcular los coeficientes de la transformación se devuelven en una matriz P, ahora de tamaño 2×4 :

$$P = \begin{pmatrix} A & B & C & D \\ a & b & c & d \end{pmatrix}$$

En las transparencias podéis ver cómo plantear el sistema lineal a resolver para obtener los coeficientes de esta transformación. Es muy similar al del caso afín, con la diferencia de que la matriz M del sistema tiene una columna adicional con el producto $x_k \cdot y_k$. Una vez construida la matriz M resolver para las coordenadas u (obteniendo A,B,C,D) y para v (a,b,c,d). Agrupar estos coeficientes como las dos filas de la matriz P (2×4) que devuelve la función. [Adjuntad el código de la función.](#)

Usad la función que habéis escrito para obtener la transformación no lineal que pase de coordenadas origen $x=[263,407,680,1]; y=[306,279,800,800];$ a las coordenadas destino $u=[92,511,498,173]; v=[232,313,766,752];$ Usando `show_mat()` [volcar la matriz P con los coeficientes encontrados para la transformación directa \(x,y \$\rightarrow\$ u,v\).](#)

A continuación modificaremos `convertir(x,y,P)` para que maneje también el caso no lineal, con matrices P de tamaño 2×4 (A,B, ..., a, b, ...) en vez de las matrices 3×3 del caso lineal. Se podría usar un parámetro adicional para indicar si estamos o no en el caso lineal, pero es más elegante hacerlo automáticamente comprobando el tamaño de P. Para ello usaremos la función `numel()` que devuelve el nº de elementos de una matriz:

- Si detectamos que P tiene 9 elementos estamos en el caso lineal (matriz 3×3) y se ejecutaría el código que ya tenemos.
- En el caso no lineal (P es 2×4) construimos una matriz ($4 \times N$) cuyas filas son sucesivamente 1's, las x's, las y's, terminando en la 4ª fila con el producto de x's e y's. Multiplicando P (2×4) por dicha matriz ($4 \times N$), se obtiene una nueva matriz cuyas filas son respectivamente las coordenadas transformadas u y v.

[Adjuntad código de vuestra función convertir\(\) modificada.](#)

Como en el caso lineal, si habéis calculado correctamente la matriz P , el resultado de hacer `[U,V]=convertir(x,y,P)`; debe ser idéntico a las coordenadas u,v destino especificadas. [Adjuntad los vectores con las diferencias entre las coordenadas \$u,v\$ originales y las \$U,V\$ obtenidas al aplicar `convertir\(\)` a las coordenadas origen \$\(x,y\)\$.](#)

Queremos ahora deformar una imagen usando esta transformación no lineal. En principio `warp_img(im,iP)` debería funcionar, ya que como la función usa `convertir()` para calcular las coordenadas de origen, usará automáticamente la transformación no lineal si detecta que iP es una matriz 2×4 en vez de 3×3 . Lo único que hay que hacer es determinar esta matriz iP correspondiente a la transformación inversa. Aquí no es posible calcular P^{-1} (ya que P es una matriz 2×4 que no tiene inversa) pero podríamos hallar iP como hicimos en el caso lineal usando `get_nolineal` con las coordenadas de origen y destino intercambiadas. [Dad la matriz \$iP\$ \(\$2 \times 4\$ \) obtenida de esta forma.](#) Comprobad que si completamos el proceso de conversión de ida y vuelta: $(x,y) \rightarrow P \rightarrow (U,V) \rightarrow iP \rightarrow (X,Y)$ debemos recuperar en (X,Y) las coordenadas de partida originales. [Adjuntad vector con las diferencias entre las coordenadas \$X,Y\$ recuperadas y las \$x,y\$ originales.](#) Usando esta matriz iP llamad ahora a `warp_img()` para deformar la imagen del fichero "foto.jpg". [Adjuntad una captura del resultado.](#)

El problema es que esta iP no es la verdadera transformación inversa. En el caso lineal, para cualquier coordenada origen (x,y) , aplicar la transformación directa P y luego su inversa P^{-1} nos devuelve a las coordenadas originales, ya que $P^{-1} \cdot P$ es la identidad. En el caso no lineal esto solo se cumple para los puntos que definen la transformación (como hemos verificado antes). Volved a probar a aplicar primero la transformación P y luego iP pero ahora partiendo de las coordenadas $(200,100)$. [Volcad las coordenadas \$\(X,Y\)\$ a las que "regresamos" \(que ya no son las mismas\).](#)

Esto nos indica que, al contrario que en el caso lineal, la transformación iP obtenida con `get_nolineal(u,v,x,y)` no es la inversa de la transformación original. De hecho, en una transformación no lineal la inversa puede ser una transformación de un tipo distinto a la original y es posible que no pueda representarse usando estas matrices 2×4 . Al no conocer la transformación inversa, si queremos deformar una imagen usando la transformación no lineal P habría que escribir una función implementando un warping "directo": `function im2=warp_directo(im,P)`. Para ello:

- Determinar el tamaño de im ($N \times M$) y reservar imagen $im2$ del mismo tamaño ($N \times M \times 3$) para guardar la imagen deformada.
- Barrer las coordenadas origen ($y=1:N$, $x=1:M$) y en cada paso:
 - Usando `convertir` aplicarles la transformación directa usando la P original y redondear usando `round()` las coordenadas (u,v) obtenidas.
 - Si las coordenadas (u,v) caen dentro de la imagen original ($N \times M$) poned en la posición (v,u) de la imagen $im2$ los contenidos del píxel origen (x,y) de la imagen original. En caso contrario no hacer nada.

[Adjuntad código de la función `warp\_directo\(\)` y una captura del resultado.](#)
[Haced zoom sobre la imagen obtenida. ¿A qué se creéis que se deben los píxeles negros que aparecen en la imagen resultado?](#)